God's number is at least 20: proving it in Rocq

Laurent Théry

INRIA, Stamp Team
Laurent.Thery@inria.fr

Abstract. God's number, the largest number of face turns needed to solve a Rubik's cube, is twenty. We describe a proof in the Rocq prover of the lower half of that statement: one position, the superflip, cannot be solved in nineteen moves. The search is pruned by a table of two billion entries, every one of them checked inside the prover.

The sources are at `github.com/thery/DoubleCover`, under `code/Rubik`, with a `README.md` saying what is in them and how to run it.

Keywords. Rubik's cube, God's number, formal proof, Rocq, pruning table.

1 The problem

A Rubik's cube is built from twenty-six small cubes: **eight corner pieces** with three stickers each, **twelve edge pieces** with two, and **six centre pieces** with one. The centres are attached to the core. They spin in place but never travel, so they are the frame that everything else is measured against: the white face is wherever the white centre is. A face turn moves four corners and four edges, and nothing else.

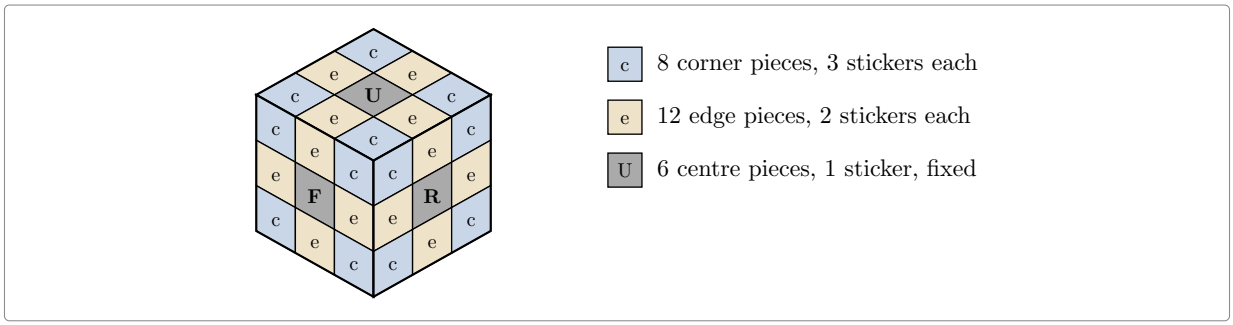


Figure 1: The three kinds of piece. Every face shows four corner stickers, four edge stickers and one centre.

Not every arrangement of the pieces can be reached by turning faces. The number of arrangements that can be reached is

$$8! \cdot 3^7 \cdot 12! \cdot 2^{11}/2 = 43\ 252\ 003\ 274\ 489\ 856\ 000 \approx 4.3 \cdot 10^{19},$$

which reads: the eight corners in any order ($8!$), each twisted one of three ways except that the last is forced by the other seven (3^7); the twelve edges in any order ($12!$), each flipped or not with the last again forced (2^{11}); and a final halving, because corners and edges cannot be rearranged independently of each other.

Three details of the pieces matter later. They are named here.

Each corner has exactly one sticker of the top colour or the bottom colour. That sticker can be in three places on the corner: on top, or on one of the corner's two sides. Which of the three is the corner's **twist**.

Each edge has a right way round. Put back the other way round, it shows its two colours the wrong way about. That is the edge's **flip**.

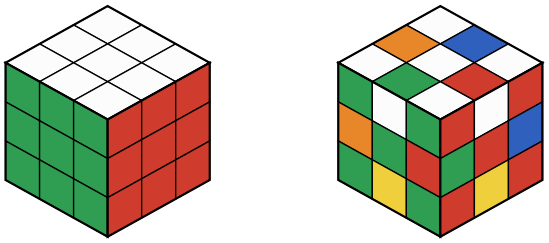
Four of the twelve edges belong in the middle layer, the **slice** between the top and the bottom face. A move can send them to any of the twelve edge slots. Which four slots they are in is the third thing to follow.

Turning one face is a *move*, and a half turn counts as one move just like a quarter turn. Every scramble can be undone; the question is how many moves the worst scramble needs. That number is called **God's number**.

In 2010 Rokicki, Kociemba, Davidson and Dethridge showed that it is **20** [1]. The answer comes in two halves, and they are not equally hard.

- **Twenty moves are always enough.** This is the huge half: every one of the 43 quintillion states has to be accounted for. It took about a billion seconds of processor time, donated by Google, after the states had been grouped into 55 882 296 families.
- **Twenty moves are sometimes needed.** For this it is enough to point at one scramble and show it cannot be solved in 19.

This note is about the second half, and about one scramble: the **superflip**, drawn in Figure 2 beside a solved cube. Every corner sticker is where it belongs, and every edge is in its own place but turned over, so it shows the colour of the face beside it. Turn the whole cube in your hands, or look at it in a mirror, and the same pattern comes back: the superflip is one of the rare positions that all 48 ways of looking at a cube leave unchanged, and that will matter later. A 20-move solution for it is known. What has to be proved is that no solution of 19 moves exists.



solved the superflip

Figure 2: A solved cube, and the superflip.

That is still a big computation. There are 18 possible moves at each step, so 19 moves means something like 18^{19} words, far more than could ever be listed. Nobody searches like that: the search is cut short using a precomputed table, and that is where both the interest and the difficulty lie.

Why do it in a proof assistant at all, here the Rocq prover [2] (the system formerly called Coq)? Because then the result no longer depends on a search program being right. What is checked instead is a proof, verified by a small kernel, starting from the definition of the cube itself.

2 The cube as permutations

The cube is easier to reason about if we stop thinking about it as a solid object. Only the coloured stickers matter. There are six faces of nine stickers, and the six centre stickers never move relative to each other, so a move is just a rearrangement of the **48 remaining stickers**. Number them 0 to 47, as in Figure 3 and Figure 4: eight to a face, taken left to right and top to bottom with the centre skipped, so up gets 0–7, left 8–15, front 16–23, right 24–31, back 32–39 and down 40–47. These are the numbers the sources use, which is what makes the definition of a move checkable against a picture.

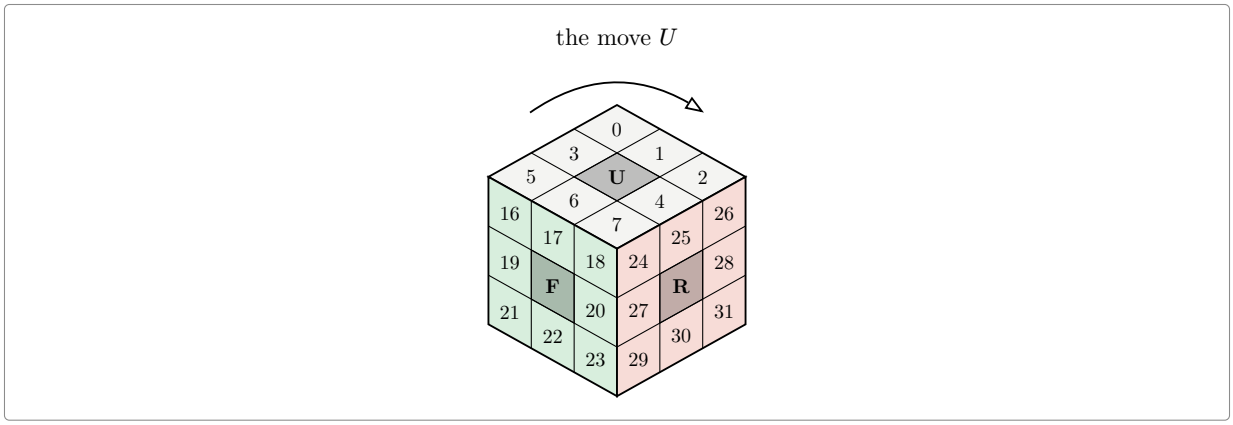


Figure 3: Three of the six faces, and the turn of the top one.

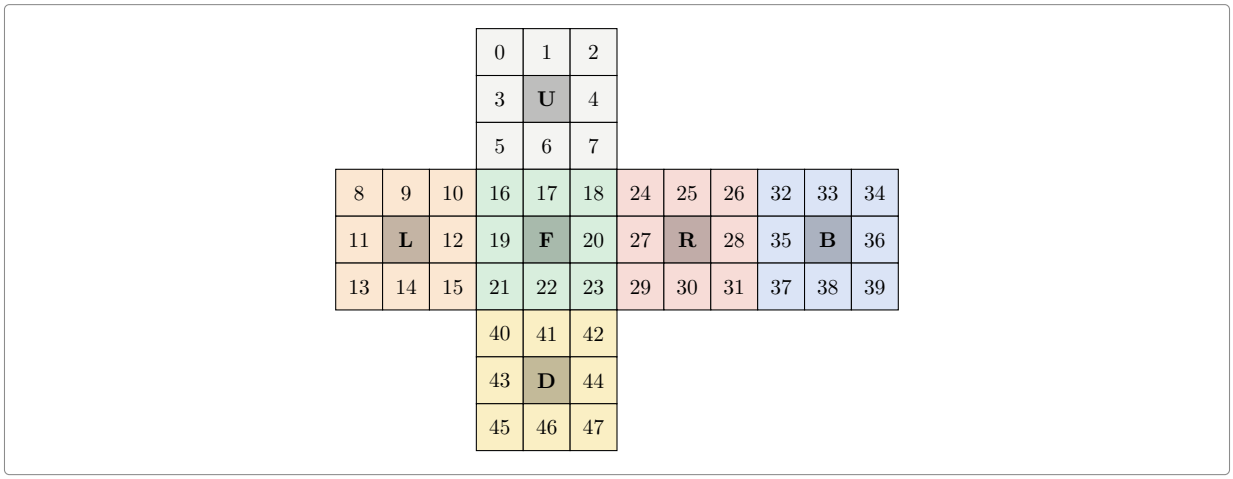


Figure 4: The cube unfolded, with all forty-eight places numbered.

With the places numbered, a move is written down by saying, for each place, where the sticker sitting there goes. Turning the top face clockwise carries the sticker in corner 0 to corner 2, the one in 2 to 7, the one in 7 to 5 and the one in 5 back to 0. That is a four step cycle, written (0 2 7 5). The four edge stickers of that face do the same, (1 4 6 3). But the turn does not only move the top face: it also carries the top row of each side face round to the next one, front to left, left to back, back to right, right to front. That is three more cycles, (8 32 24 16) and its two companions. Figure 5 shows the whole of it, each square saying which sticker sits there afterwards: the top row of the left face holds 16, 17, 18, the stickers that came round from the front. The other forty stickers are untouched, and the six centres never move at all.

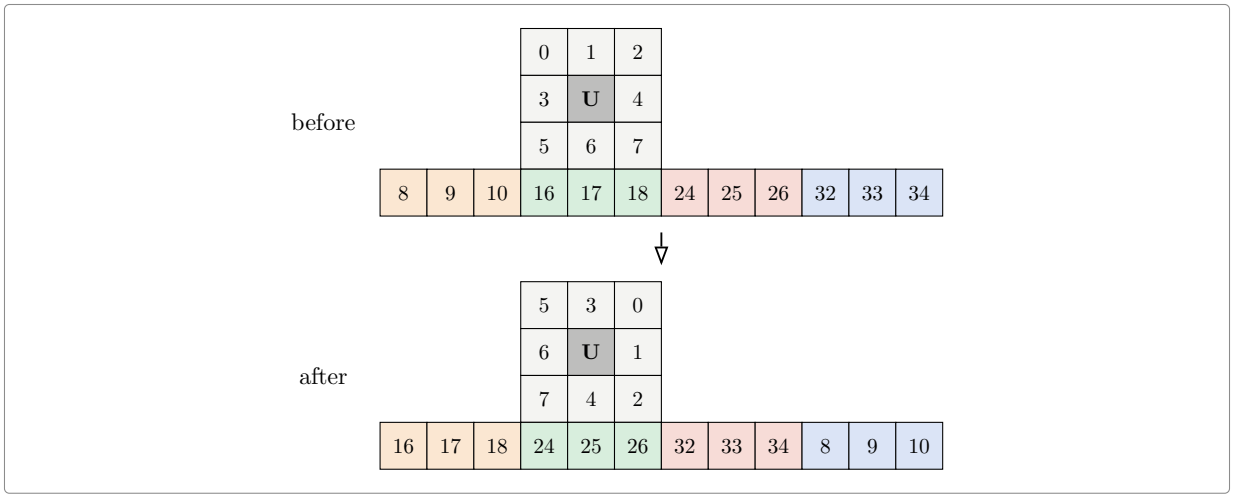


Figure 5: Everything a clockwise turn of the top face moves.

Six clockwise quarter turns generate everything: up, right, front, down, left and back. Each of them can also be done twice or backwards, which gives the **eighteen moves**

$$\begin{array}{cccccc} U & R & F & D & L & B \\ U^2 & R^2 & F^2 & D^2 & L^2 & B^2 \\ U^{-1} & R^{-1} & F^{-1} & D^{-1} & L^{-1} & B^{-1} \end{array}$$

A scramble is a product of moves, for instance $RUR^{-1}U^{-1}$, a word of length 4. The set of all scrambles is a group G: the **cube group**. Solving a scramble in d moves means writing it as a word of d moves, so "solvable in at most d moves" says exactly that the scramble lies in the **ball of radius d** around the solved cube. God's number is the largest distance that occurs, the diameter of that ball structure.

2.1 What this looks like in Rocq

The development is built on **mathcomp** [3], a large library of formalised mathematics that already knows about permutations, groups and products. The cube file is then just a transcription of the paragraphs above, and it is short:

```
Definition facelet := 'I_48.

Definition Umove : {perm facelet} :=
  cyc [:: 0@; 2@; 7@; 5@] * cyc [:: 1@; 4@; 6@; 3@] *
  cyc [:: 8@; 32@; 24@; 16@] * cyc [:: 9@; 33@; 25@; 17@] *
  cyc [:: 10@; 34@; 26@; 18@].
  (* ... and five more, one per face ... *)

Definition faces : seq {perm facelet} :=
  [:: Umove; Rmove; Fmove; Dmove; Lmove; Bmove].
Definition moves : seq {perm facelet} :=
  flatten [seq [:: g; g ^+ 2; g ^-1] | g <- faces].
Definition G : {group {perm facelet}} := <<Sset>>.
```

Word by word:

- `'I_48` is the type of the whole numbers **below** 48, so the places are numbered **0 to 47** and not 1 to 48, everywhere in the sources and in the pictures of this note.
- `{perm facelet}` is the type of **permutations** of those places: a way of sending each place to a place, no two of them landing on the same one. That is exactly what a scramble is.
- `cyc [:: 0@; 2@; 7@; 5@]` is the **cycle** that sends 0 to 2, 2 to 7, 7 to 5 and 5 back to 0, leaving the other forty-four places where they are. The `@` is local notation turning a plain number into a place.

- * composes two permutations, so `Umove` is the five cycles of Figure 5 done together, and `g ^+ 2` and `g ^-1` are the same turn done twice and undone.
- `seq` is a list, and `faces` is the list of the six clockwise quarter turns. `moves` runs through it and keeps three moves per face, which is the eighteen.
- `<<Sset>>` is the group generated by a set: everything reachable by composing moves, which is the cube group.

The superflip is written down the same way, as the twelve swaps that exchange the two stickers of each edge:

```
Definition Spcyc : seq (seq facelet) :=
  [:: [:: 1@; 33@]; [:: 3@; 9@]; [:: 4@; 25@];
      (* ... nine more, one per edge ... *) ].

Definition superflip : {perm facelet} := \prod_(l <- Spcyc) cyc l.
```

That makes it a permutation of the stickers, but by itself it says nothing about the cube. A permutation is a legal position only if the faces can actually be turned to reach it, that is, only if it lies in G. Nothing in the definition above gives that, and it has to be proved.

The proof is one equality. On the left, the superflip as just defined. On the right, a word of twenty moves:

$$U\ R^2\ F\ B\ R\ B^2\ R\ U^2\ L\ B^2\ R\ U^{-1}\ D^{-1}\ R^2\ F\ R^{-1}\ L\ B^2\ U^2\ F^2$$

Both sides are permutations of the 48 stickers. Each is written out as the list of the 48 places it sends each place to, and the equality becomes one comparison of two lists. Every letter on the right is one of the eighteen moves, so the superflip lies in G. That same word is also what gives the upper bound of 20 for this one position.

Nothing here is assumed. There is no axiom stating what a cube is, and a reader who wants to check the model only has to compare the six lists of cycles against Figure 4. After this file, stickers are never mentioned again.

3 How the search works

Two classical ideas make the search possible.

3.1 An estimate that is never too big

The idea is not new, and neither is the way the estimate is obtained. A depth-first search that deepens step by step and prunes on an estimate that never over-estimates is Korf's IDA* [4]; taking the estimate from a table of exact distances in a simplified version of the puzzle is a *pattern database* [5], and Korf solved the cube optimally with three of them [6]. The summary used here is Kociemba's, from his two-phase solver [7].

Suppose we have a way of estimating, for any scramble, how many moves it needs : an estimate that is never larger than the truth, and that changes by at most one when a move is made. Call it h.

Such an estimate is a pair of scissors. Walk down the tree of moves, keeping track of how many are left. If the estimate for a position is 20 while only 18 moves remain, that whole branch can be dropped: it cannot possibly reach the solved cube in time. Figure 6 shows the picture: below every position the table is consulted, and a branch whose estimate exceeds the moves still available is abandoned without ever being explored. The estimate is allowed to be too small, which only means less cutting; it is never allowed to be too large.

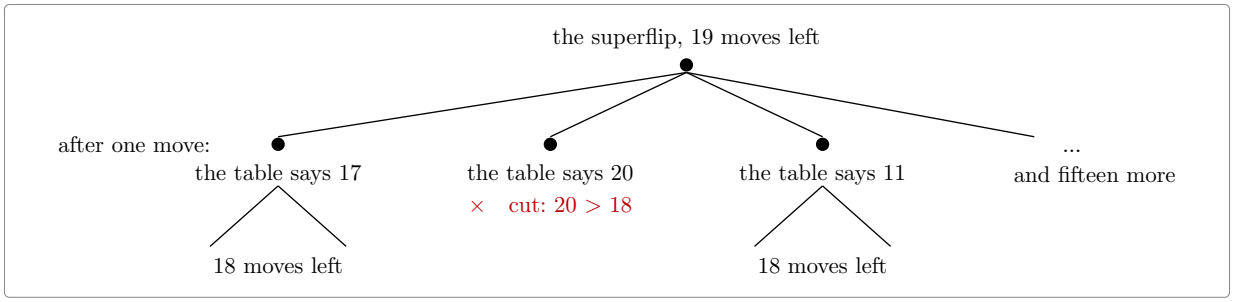


Figure 6: The search, and its scissors.

3.2 Where the estimate comes from

The trick is to forget most of the cube. Keep only part of the information, say how the corners are twisted and where the four middle-layer edges sit, and call what is left a **summary**. Many different scrambles share a summary. Moves act on summaries just as well as on cubes, and there are few enough summaries that a computer can work out, once and for all, the exact distance from the solved summary to every other summary. That table of distances is the estimate: a scramble needs at least as many moves as its summary does.

And here is where the numbering of the stickers earns its keep: a summary is read straight off them. Figure 7 shows the three questions it asks. A corner has one sticker belonging to the up or down face, and that sticker sits in one of three places, which is 0, 1 or 2. An edge is either the right way round or turned over, which is 0 or 1. And the four edges of the middle layer occupy four of the twelve edge slots; the figure shades them on the two visible faces, where three of the four can be seen. Nothing else about the position is recorded.

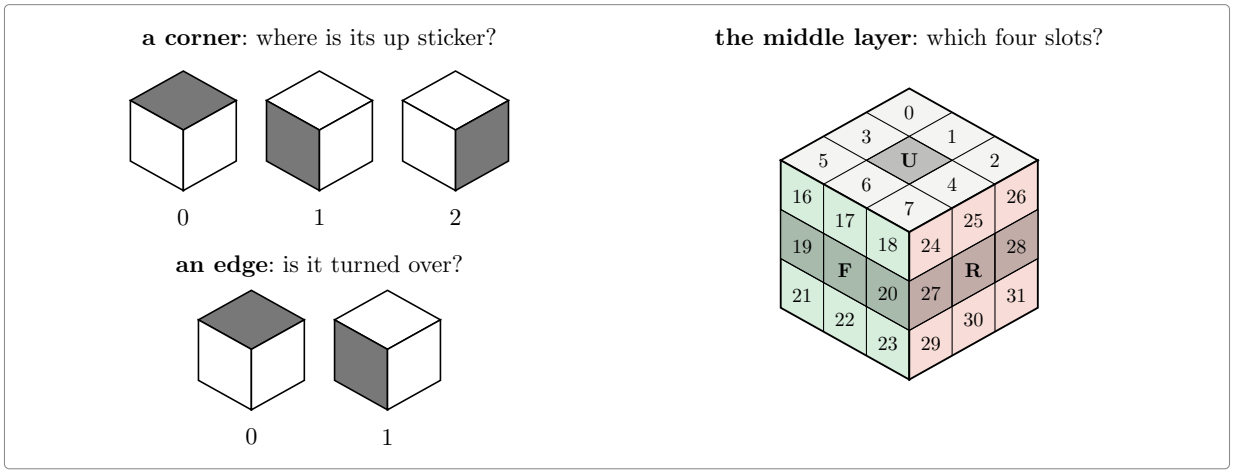


Figure 7: The three questions a summary asks.

The summary is the product of the three answers:

summary	**values**	
how the eight corners are twisted	2 187	$= 3^7$
how the twelve edges are flipped	2 048	$= 2^{11}$
where the four middle-layer edges sit	495	4 places among 12
the phase 1 summary, all three together	**2 217 093 120**	

The name comes from Kociemba's solving algorithm, which works in two phases. Its first phase has to bring exactly these three answers to zero, and the summary is what it watches while doing so. The algorithm itself plays no part here. Only the name is borrowed, because it is the one everybody uses for this table.

Two billion is small next to 43 quintillion: **every summary stands for exactly 19 508 428 800 real scrambles** (that is the division, and it comes out even). The table records, for each summary, its distance from the solved summary. No entry in it is larger than 12, that being how deep the deepest summary lies rather than the value of any position in particular. So four bits hold one entry, and the whole table is **1.18 GB**.

3.3 And why a failed search is a proof

The search answers "maybe solvable in d moves" or "no". The **no** is the trustworthy side: it means every branch was cut, either because the depth ran out or because the table said so. So a negative answer to "is the superflip solvable in 19 moves?" is exactly the theorem wanted.

Better still, the answer does not depend on the table being right. If an entry of the table were too small, the search would merely cut less and take longer. Only the two local properties matter: the solved summary has distance 0, and one move changes the value by at most one. That is what makes the whole thing formalisable at a sane cost: a table with two billion entries never has to be proved correct, only checked.

4 The search in Rocq

The general principle is one file of about a hundred lines, `Search.v`, which does not mention the cube at all. It takes a group, a set of moves, an estimate h, and the two assumptions:

```
Hypothesis h1    : h 1 = 0.
Hypothesis hstep : forall g m, m \in S -> h g <= (h (g * m)).+1.

Fixpoint search (d : nat) (g : gT) : bool :=
  (h g <= d) &&
  ((g == 1) ||
   (if d is d'.+1 then has (fun m => search d' (g * m)) Sseq else false)).

Corollary searchN d g : search d g = false -> g \notin ball S d.
```

Again word by word:

- `gT` is the group the file works in. It is a variable, so nothing here is about the cube; the cube is what it gets instantiated with later.
- `1` is the unit of that group, which for the cube is the **solved** position. So `h 1 = 0` says the estimate of a solved cube is zero, and `g == 1` asks whether the search has arrived.
- `Sseq` is the list of moves, the eighteen of them, in the order the search walks over them. `S` is the same thing seen as a set.
- `g * m` is the position `g` followed by the move `m`. The order takes some getting used to: mathcomp applies permutations on the right, so `(g * m) f` is `m (g f)`, and the product reads left to right like a sequence of moves rather than right to left like a composition of functions.
- `h g <= d` is the cut, and `(h (g * m)).+1` is the estimate after a move plus one, which is the assumption that one move changes the estimate by at most one.
- `has (fun m => search d' (g * m)) Sseq` tries every move with one fewer move available, and answers as soon as one of them succeeds.

The last line is the contract of the whole development: **if the search returns false, the position is not within d moves.**

That is a theorem, proved once, about any estimate satisfying the two assumptions. Everything else in the development exists either to discharge those two assumptions for the real table, or to make the search fast enough to run.

What the table has to satisfy, and what it does not. `Coord.v` asks for three things and checks two:

```
Variable coord : {perm facelet} -> X.
Variable act   : X -> {perm facelet} -> X.
Hypothesis coordM : forall g m, coord (g * m) = act (coord g) m.

Variable D : X -> nat.
Hypothesis D0    : D (coord 1) = 0.
Hypothesis Dstep : forall x m, m \in Sset -> D x <= (D (act x m)).+1.
```

`coord` is the summary of a position, `X` being whatever the summaries happen to be. `act` is how a move acts on a summary directly, without going back to the position it came from: for the phase 1 summary it is two lookups in a move table. `coordM` is what makes the pair worth having, and it is the one thing proved about the summary itself. It says that summarising after a move gives the same answer as acting on the summary. So the search never computes a summary from a position. It carries the summary beside the position and brings it up to date one move at a time, at the cost of a lookup. It still carries the position: the summary cannot say whether the cube is solved, only the position can.

Put beside the `search` of `Search.v`, the search that actually runs has this shape. Names are simplified and the machine-integer details left out; the real one is `searchz3` in `Farp1.v`:

```
Fixpoint search (d : nat) (g : gT) (x : summary) (p : move) : bool :=
  (D x <= d) &&
  ((g == 1) ||
   (if d is d'.+1
    then has (fun m => search d' (g * m) (act x m) m) (allowed p)
    else false)).
```

Two things travel down the tree instead of one, and each is used for exactly one job:

- `D x <= d` is the cut. It reads the table at the summary `x`, and never looks at the position.
- `g == 1` asks whether the cube is solved. It reads the position `g`, and never looks at the summary.
- `g * m` moves the position and `act x m` moves the summary, side by side, one move at a time. That step is `coordM` being used, and it is why the summary never has to be recomputed from the position.
- `p` is the move just played, and `allowed p` is the list of moves the rules permit after it. That is where redundant sequences are dropped.

The two conditions. `D` is the estimate. It takes a summary and gives back a number, and that number is read from the table. `D0` and `Dstep` are everything asked of it. `D0` says the solved cube gets zero. `Dstep` says that one move lowers the estimate by at most one.

That is a weak demand, and it is worth seeing how weak. The table is never proved to hold the true distance to the solved cube. A table of zeros passes both conditions. It would prune nothing, and the search would run for ever, but it would not make the search give a wrong answer.

How the two conditions are checked. Look again at `Dstep`. Unlike `D0`, it does not mention `coord` at all. It speaks of every value `x` in `X`, and not only of those values that are the summary of a real position. That is deliberate, and it is what makes the check possible. `X` is a finite set of 2.2 billion values, so the check runs over all of them, and it never has to know which of them come from a cube. `D0` is then one lookup, and `Dstep` is one sweep: 2.2 billion summaries, eighteen moves each, one comparison apiece. Those sweeps are what the *certificate* files do: `FsmChk.v`, `FsrChk.v`, `SlrChk.v`, `P1TsChk.v` and `Farp1chk.v`, listed again at the end of this note, each ending in its own `Qed`.

Nothing in this asks where the table came from. It can be written by any program in any language. Here an OCaml generator writes it out as Rocq source, and the two conditions are checked on it afterwards.

Three cuts at the top of the tree. They are three different arguments and it is worth keeping them apart. The first two apply once each, to the first move and to the second. The third applies at every move from the third on.

The first move: eighteen become two. The superflip looks the same from every angle. There are 48 ways of putting a cube back into the space it came from: any of the six faces can be turned to the top, each of them in four positions, which makes twenty-four, and each of those seen in a mirror as well. Relabelling the superflip's stickers by any of the 48 gives the superflip back again. So the first move may be taken to be U or U^2, and the other sixteen need not be tried.

The second move: eighteen become fifteen. The three that turn the top face again are dropped. Turning the top face twice running merges into a single turn, so those give a word of nineteen moves or fewer, and the proof covers shorter words at a smaller depth rather than by a search of their own.

The three that turn the **bottom** face look just as droppable, and are not. It is worth saying why, because it is the case everyone gets wrong. The two turns commute, so UD can be rewritten DU; but the first move is already pinned to the top face by symmetry, and turning the cube over to bring that D back to the top gives UD once more. It is a fixed point of both rewritings, so neither rewriting removes it. Reid's proof meets the same case and pays for it elsewhere: he keeps the bottom-face second move, and uses the symmetries that fix the pair of opposite faces to cut his **third** move instead.

Our own OCaml program had this wrong. It dropped the three bottom-face second moves and so searched 24 prefixes where it had to search 30. Nothing about the program looked wrong: it ran for hours, it exhausted its tree, and it reported that no solution of length 19 exists, which is the answer we expected. The error only came out when the same reduction had to be proved in Rocq, and the proof of the bottom-face case could not be written. This is the whole argument for proving a search rather than trusting it. A cut that is too greedy does not make a search fail. It makes it faster, and it makes it agree with you.

From the third move on: eighteen become fifteen or twelve. This one is not about the superflip and not about the top of the tree. It is one rule, applied at every node. A shortest word never turns the same face twice running, since the two turns merge into one; and of two opposite faces it never uses both orders, since UD and DU give the same position, so fixing one order loses nothing. At a node whose last move was on the top, right or front face, both halves of the rule bite and **twelve** moves are left. At a node whose last move was on the bottom, left or back face, only the first half bites, because the opposite pair has already been cut in the other order, and **fifteen** are left.

Applying that rule to the third move is not an extra assumption. The word after the second move is reduced like any other, and the proof already knew it; the search had simply been throwing the fact away and trying all eighteen.

None of the three is obvious, and they interact. The bottom-face second move survives only because the rule used from the third move on would otherwise cut the same pair of opposite faces twice, once by symmetry and once by the order convention. Arguments of that shape are easy to get wrong and hard to test, since getting one wrong makes the search faster and leaves the answer looking the same. This is what Rocq is for here. Every cut has to be proved before the search is allowed to use it, so a cut that does not hold is refused rather than rewarded, and cuts that would otherwise be too delicate to trust can be taken.

So the depth-19 search becomes $2 \times 15 = 30$ searches of depth 17. The 30 are packed into **seventeen files**, `Runp1_03.v` to `Runp1_17.v`, one per second move and numbered by it, each holding both first moves.

Two of them run far longer than the rest, and they are the two whose second move turns the bottom face, D and D^{-1}, where the rule above leaves fifteen branches and not twelve. Each is cut in two, one

first move to a file: `Runp1_09a.v` and `Runp1_09b.v` in place of a single `Runp1_09.v`, and `Runp1_11a.v` and `Runp1_11b.v` in place of `Runp1_11.v`. Fifteen second moves, two of them split, makes seventeen files. Without the split, the run would finish long before those two files did, and would have to wait for them.

That is how the work is spread over the cores of a machine: seventeen files, seventeen `Qed`s, nothing shared.

5 What had to be optimised

The first version worked and was far too slow. Closing the gap between "runs" and "finishes" took a series of changes, each one measured before and after.

Counting in unary is the enemy. The numbers used by the mathcomp library are Peano numbers: 5 is literally the successor of the successor of the successor of the successor of the successor of zero, so adding n costs n steps and comparing costs as much again. Rocq also offers machine integers, 63 bits wide with the missing bit going to the garbage collector, and **persistent arrays** of them [8], both of which cost what the hardware costs. That is also how the table is stored: fifteen of its four-bit entries to a machine integer. Measured here: a machine-integer operation takes about 0.05 microseconds and a Peano-number operation about 1 microsecond, and converting between the two costs about 0.07 microseconds *per unit*, so that converting the number 495 costs 36 microseconds all by itself. Rewriting the inner loop so that indices, table values and the depth comparison never leave machine integers gave:

operation	before	after
reading the summary of a position	32.0 µs	**0.10 µs**
applying a move to a summary	4.60 µs	**0.12 µs**
reading one entry of the packed table	2.99 µs	**0.13 µs**

`and` and `or` are function calls. Rocq evaluates both sides of a boolean test before combining them, so a guard written "either the value is out of range, or the expensive check holds" runs the expensive check on *every* value. Rewritten as a nested `if`, it runs only on the values that get past the guard. Two otherwise identical certificates, one of each shape: **719.7 s against about 80 s**. The same mistake in the search cost a further factor of 27.8 on one guard. Every guard in the development is a nested `if` now.

The search itself, twelve times faster. Before the list of changes, here is what the search of `Farp1.v` actually carries at each position, since none of them make sense otherwise. Four things:

- `a`, the position itself as a 48-entry array saying where each sticker went. It is used for one purpose only, to ask "is this the solved cube?", and a fresh one is built for each child by composing `a` with the move's own array.
- `x`, the three views: for each of the three, the pair of numbers that is its summary, the corner twist and the flip-and-slice index. A move takes each pair to another pair by two lookups in a move table.
- `p`, which face was turned last, from which the list of moves worth trying next is read off.
- `d`, how many moves are left.

The estimate is the largest of **nine** lookups: three tables, at each of the three views. Now the chain of seven versions in `Fast.v`, each proved equal to the one before, so that no trust is transferred. Measured on one piece at depth 14:

version	seconds	what it removes
the original	70.0	

version	seconds	what it removes
2	39.1	Peano arithmetic inside the loop: move indices, the depth test and the list of allowed moves all become machine integers, computed once
3	16.5	building the child's 48-entry array before looking at the estimate, when the estimate then rejects the child
4	13.4	comparing all 48 entries against the solved position when the first difference already settles it
5	9.7	the last of the nine lookups once one of them already exceeds the moves left
6	8.0	computing the summaries of all three views when the first view already cuts
7	**5.9**	keeping `a` up to date at every position: the list of moves is carried instead, and the position rebuilt from it only for the solved test
	11.9x	

Four of those six steps are the same mistake in different clothes: work being done that a lazier evaluator would have skipped.

Three views of the same position. Rotating the whole cube about a corner axis gives the same position seen differently, and its summary is then a different entry of the same table. Each of the three views therefore yields a lower bound on the number of moves left, so the largest of them is a lower bound too, and never a smaller one than any single view gives. Three times the lookups at each position, in exchange for a sharper cut and so a smaller tree. This is standard practice in cube solvers, Kociemba's included; what is new here is only that the three views are proved to be legitimate.

How the table is written down costs more than the table. The phase 1 table is emitted as Rocq source, one file per block of 2 097 152 entries, 71 of them. Written as a list, a block is a term: two million nested applications of the list constructor, each holding a machine integer. That term is what the `.vo` stores and what `Require` loads, and it is far larger than the 17 MB of data in it. Written instead as an array literal, that is, as a definition whose body has already been evaluated to a primitive array, the `.vo` holds one compact block of memory. Measured on the same 2 097 152 entries:

the block, written as	its `.vo`	loaded
a list	37.8 MB	877 MB
an array literal	**6.0 MB**	**281 MB**

Over all 71 blocks that is 21.5 GB against **5 GB**, and it is what allowed nine parallel workers instead of two on a 62 GB machine.

Folding the table by symmetry. The summary is built around the up-down axis: the twist counts where each corner's up-or-down sticker sits, and the slice says where the four edges between the top and bottom faces are. So a symmetry that leaves that axis in place turns a summary into another summary, while one that tips the cube onto another axis does not act on these summaries at all. Sixteen of the 48 keep the axis, and they sort the 1 013 760 flip-and-slice values into **64 430 families**, a factor of **15.73**. Two values in the same family are the same distance from solved, so one entry per family is enough: a lookup first replaces the value by its family's representative, carrying the twist through the same symmetry, and then reads a table 15.73 times smaller.

That is a different use of symmetry from the three views above, and the two are worth telling apart. The three views ask the **same** table three questions and keep the largest answer, which sharpens the estimate and cuts the tree. The fold asks the **same** question of a **smaller** table, and the estimate does not change at all. Symmetry-reduced tables of this kind are standard in cube solvers; what the development adds is a proof that the folded table still satisfies the two conditions.

This is where the weakness of the two conditions pays. Nowhere does the proof say that the folded table holds distances. It says the table passes `D0` and `Dstep`, and the search needs nothing more. Had the conditions been written to demand true distances instead, the fold would have had to be shown to preserve them, which is a harder statement about the sixteen symmetries and about what sharing an entry between two summaries does to it. None of that has to be faced. The check is run on the folded table just as it was on the flat one, and it is the same check.

Nor would demanding more have cost more. A table of true distances is recognised by the same single sweep: it holds them exactly when it passes the two conditions and, at every summary but the solved one, some move lowers the value by exactly one. Asking for less does not buy a cheaper check. It buys the freedom to hand the search a table like this one.

The fold costs the search, measured at 1.61 times slower at depth 16, and pays everywhere else: a search worker drops from 4.15 GB to **0.85 GB**, so all the pieces now run at once instead of in two waves, and checking the table drops from about 5.4 processor hours to **1.35**.

What remains. The same search written in OCaml is about three times faster. Both were run at radius 19 on the reference machine. The OCaml program visits 146 065 078 152 positions in 26.4 processor-hours, which is 0.65 microseconds a position; Rocq takes 87.6 processor-hours over the same tree, which is 2.16. A factor of **3.3**.

That the two walk the same tree is not assumed. Dividing each of the seventeen Rocq pieces by the positions its OCaml counterpart visited gives between 1.98 and 2.52 microseconds, across pieces that differ in size by a factor of two. A Rocq search cutting differently anywhere would show up as scatter there, and there is none.

So the run takes a night because the tree has 146 billion nodes in it, not because the prover is slow. In OCaml the same tree still costs 26 processor-hours.

6 The files

Forty-six hand-written Rocq files. What each group does. The sources carry their own `README.md`, which lists the same files, the scripts beside them, and how to run the whole thing.

the cube, as mathematics	
`Cyc.v`	cyclic permutations, built from the list of points they move
`Rubik333.v`	facelets, the six faces, the eighteen moves, the cube group
`Sym.v`, `Sym16.v`	the 48 symmetries of the cube; the 16 used by the fold
`Ball.v`	balls of radius d, and what "the diameter is at most d" means
`Diameter.v`	the superflip, its 20-move sequence, and what the upper bound would need

the search, in the abstract	
`Search.v`	the search and its contract: a false answer is a proof
`Coord.v`	any summary plus any checked table gives a legal estimate
`Root.v`	the first move, up to symmetry: U or U^2
`Searchr.v`, `Redun.v`	the rules that forbid redundant move sequences

data structures	
`Table.v`	permutations written as the table of their images
`Tabi.v`	the same on machine integers and arrays, and the bridge between
`ssrint63.v`	the machine-integer toolbox used throughout

the summaries and their tables	
`Coordfs.v`, `Coordfsi.v`	the edge-flip and slice summary, packed into 24 bits

the summaries and their tables	
`Fstab.v`, `FsTable.v`, `Fsparity.v`	its table and the checks it must pass
`Phase1.v`	the phase 1 estimate, its table and its certificate, 2215 lines
`Moves.v`	the eighteen moves and the superflip, as tables

the search on the real data	
`Farp1.v`	the three viewing angles and the search built on them, 1404 lines
`Far.v`	the assembly: the superflip is not within d moves, 1124 lines
`Fast.v`, `FastP.v`	the fast search, and the proof that it is the same search
`Runp1_03.v` .. `_17.v`	the seventeen pieces, written by a script
`Farp1main.v`	the theorem over *any* table: 8 seconds, and no data at all
`Farp1inst.v`	the same at the real table and the seventeen real runs
`Diam20.v`	and hence God's number is at least 20

The **certificates** are the files that run the checks, each with its own `Qed`. Four of them cover the move and distance tables and the second summary table: `FsmChk.v`, `FsrChk.v`, `SlrChk.v` and `P1TsChk.v`. `Farp1chk.v` checks the shape of the main table. Two more check the main table itself, split sixteen ways, and the folded table is checked by the `Fold` group, whose slices are `FoldOrbit_00.v` to `FoldOrbit_26.v`. Three of the files that hold the actual numbers are deliberately kept out of the project file, since they do not exist until the generator has run.

Outside Rocq, `ocaml/rubik_par.ml` and `ocaml/rubik_lb.ml` are the reference implementation. The Rocq search reproduces their node counts exactly, which is how a discrepancy gets caught early, and `bench/p1gen.ml` generates the tables. The `bench/` directory also keeps the experiments that settled design questions, each with its measurements.

7 The development in figures

hand-written Rocq, about the cube	**45 files, 12 725 lines**
`ssrint63.v`, a general int63 toolbox	1 308 lines
generated table sources, in the repository	about 156 000 lines
generated table sources, too big to store	about 165 MB of literals
OCaml reference programs and generators	2 749 lines
build and run scripts	1 031 lines

Positions visited by the radius-19 search, counted by the OCaml program. The Rocq search walks the same tree, and reproduces these counts exactly at the smaller depths where counting it is cheap:

	positions
the smallest piece, `Runp1_11b`	5 575 767 076
the largest piece, `Runp1_10`	10 554 835 820
all seventeen	**146 065 078 152**

The tree grows by 12.22 from one level to the next, measured between depths 17 and 19.

Building the tables costs the same whatever radius is searched afterwards. Measured end to end from a clean tree on the reference machine, a dual-socket Xeon with 62 GB and twelve physical cores:

	wall clock	**processor time**
emitting the tables and compiling them to native code	17 min 21	52 min 13
the coordinate and summary tables	22 min 46	21 min 15
the search and the estimate, over no data at all	11 min 02	30 min 45
the four certificates for the move and distance tables	56 s	2 min 27

	wall clock	processor time
the fold: twelve checks and twenty-seven slices	9 min 52	47 min 57
the shape check against the dummy table	18 s	16 s
in total	**1 h 02**	**2 h 35**

Two things that table says. The second line is **serial** — 21 minutes of processor time inside 23 minutes of wall clock — so it is the longest stage by the clock and no number of cores shortens it. And the first and fifth lines together are 100 of the 155 processor-minutes, both dominated by the OCaml compiler turning a table into native code.

8 The theorem, its cost, and what is left

The statement proved at the top of the chain is

```
Theorem superflip_p1far_real : superflip \notin ball Sset p1depth.
```

In words, the superflip is not within `p1depth` moves of the solved cube, where the depth is set by one script before the run. It has **no hypotheses left**: the six computations it rests on, namely the two summary tables, the three move and distance tables and the searches, each live in their own file behind their own `Qed`. Asking Rocq what the proof assumes reports only the primitives of its machine-integer and array interface. Nothing in the chain is admitted.

At depth 19 this says precisely that the superflip cannot be solved in 19 moves. Two lines then turn it into **God's number $\geq$ 20**, in `Diam20.v`: that file is where the two ends of the development meet, the cube being defined at the bottom of the chain and the search sitting at the top, and it first checks that the searches were indeed run at 19.

The run that proves it, measured on the reference machine, twice: once before the fold and the two reductions and once after, the same theorem both times.

radius 19, search depth 17	before	after
pieces	18	**17**
workers	9	**18**
memory per worker	4.15 GB	**0.85 GB**
wall clock	11 h 13	**6 h 36**
processor time	85 h 11	**87 h 36**

The pieces do not all take the same time. The shortest took 3 h 38 and the longest 6 h 36, read from the times at which they finished. The run ends when the longest piece ends, so the other sixteen workers are idle before that. Two of the fifteen search positions, the ninth and the eleventh, are cut in half for this reason. Were they not cut, each would take about 9 h 50, and the whole run would take that long. If the work were shared evenly over eighteen workers it would take 4 h 54. So about 1 h 40 is lost to idle workers, and to save it the eight longest pieces would have to be cut in half as well.

The two columns say that the fold and the two reductions cut the wall clock by 41% and left the processor time as it was, 3% higher. The gain in wall clock comes from the memory: a worker needs 0.85 GB, so seventeen pieces fit at once, where 4.15 GB allowed only nine. The processor time is a surprise. On the small test used while the reductions were written, the fold was 1.61 times slower and the reductions 1.86 times faster. Together that is a saving of about a sixth, and the run shows no saving at all. That small test is not to be trusted: it subtracts two large numbers, 245 processor-seconds of table loading from 391 for the whole run, to get 146 of search. The run above is the number to trust for the cost of the theorem. Why the two factors do not add up has not been measured.

Memory. A worker holds the loaded table and nothing else that grows: the search walks down and back up, so only the current sequence of moves is kept. The 4.15 GB is identical to three decimals

across the nine workers and the same at every depth, and the fold is what brings it to 0.85 and lets all seventeen pieces run at once.

What is left. The proof that God's number is at least 20 costs 87 processor-hours and one night. That is a measured cost, not an estimate. Two savings are in sight and neither is large. The first is the 1 h 40 of idle workers at the end of the run, which is a matter of how the work is shared out and not of mathematics. The second is the factor of 3.3 against OCaml, and even winning all of it would leave 26 processor-hours. The cost is the tree, and the tree is 146 billion positions. Nothing else in the chain is known to be wasteful.

And the other half of God's number, that 20 moves always suffice, is not proved here. `Diameter.v` does contain the reduction for it, stated in terms of exactly what an exhaustive search would have to supply: that the 55.9 million families cover every case up to symmetry, and that each of them is solvable in 20 moves. That computation is several orders of magnitude larger than the one this note describes.

This development was written with the help of Claude, Anthropic's coding assistant, which is recorded as a co-author of 267 of the 268 commits of `code/Rubik`.

References

1. Rokicki, T., Kociemba, H., Davidson, M., Dethridge, J.: The Diameter of the Rubik's Cube Group Is Twenty. SIAM Journal on Discrete Mathematics. 27, 1082–1105 (2013). https://doi.org/10.1137/120867366
2. The Rocq Development Team: The Rocq Prover, https://rocq-prover.org/
3. Mahboubi, A., Tassi, E.: Mathematical Components. Zenodo (2021)
4. Korf, R.E.: Depth-first Iterative-deepening: An Optimal Admissible Tree Search. Artificial Intelligence. 27, 97–109 (1985). https://doi.org/10.1016/0004-3702(85)90084-0
5. Culberson, J.C., Schaeffer, J.: Pattern Databases. Computational Intelligence. 14, 318–334 (1998). https://doi.org/10.1111/0824-7935.00065
6. Korf, R.E.: Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In: Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97). pp. 700–705 (1997)
7. Kociemba, H.: The Two-Phase Algorithm, https://kociemba.org/cube.htm
8. Armand, M., Grégoire, B., Spiwack, A., Théry, L.: Extending Coq with Imperative Features and its Application to SAT Verification. In: Interactive Theorem Proving (ITP 2010). pp. 83–98. Springer (2010)